

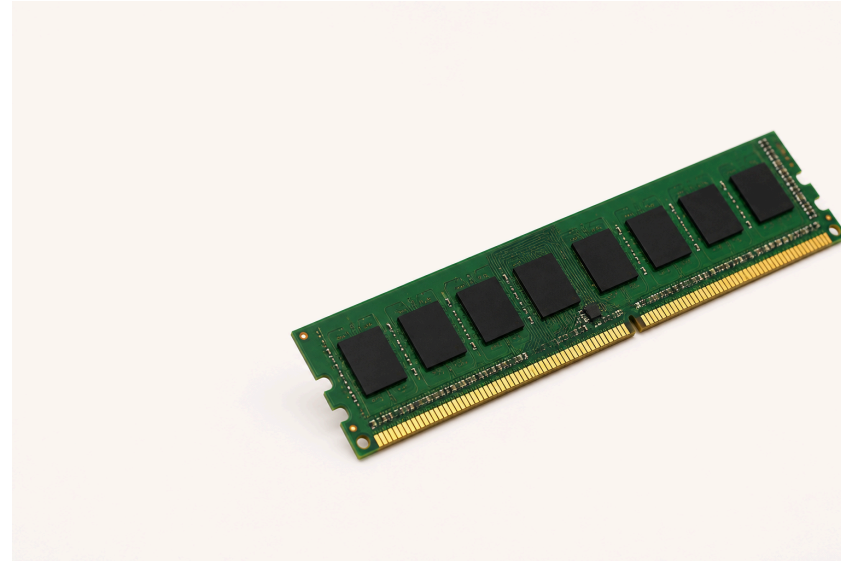
Lecture 2: Data Representation I – Bits, Bytes & Integers

Session 2 · Mon Aug 31, 2026

From Physical Memory to One Byte

That is a **RAM module** — physical hardware, many chips, not one byte.

Memory is **byte-addressable**: every address names one **8-bit byte**, and a byte is the smallest unit a C program can address on its own.



One byte = 8 bits. Today is about what fits in one, and how to work on it.

One Byte, Two Readings

Start with one byte: eight switches that are each either 0 or 1.

One byte: the bit pattern for 65



64 + 1 = 65

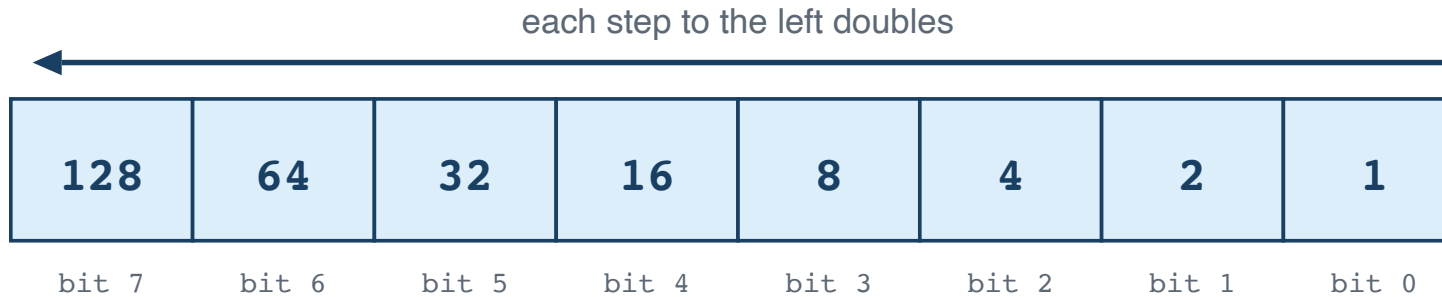
as text (ASCII): A

The computer stores only the bits; the program decides how to read them.

Counting in Binary

Why does **01000001** mean 65? Because each place is worth twice the one to its right — the same idea as decimal's ones, tens and hundreds, with 2 instead of 10.

Each place is worth twice the place to its right



Turn a bit on and you add its place value.

$$128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$$

Counting Up

Counting works exactly the way it does in decimal — when a column runs out of digits, it rolls over and carries into the next one:

0 = 0000	3 = 0011	6 = 0110	9 = 1001
1 = 0001	4 = 0100	7 = 0111	10 = 1010
2 = 0010	5 = 0101	8 = 1000	11 = 1011

Each extra bit doubles the patterns available: 8 bits give $2^8 = 256$. So an unsigned byte runs 0–255, and an N-bit unsigned value runs 0 to $2^N - 1$ — its **UMax**, modulo 2^N .

Writing the Same Number Three Ways

A number does not change when you write it differently. C lets you say which notation you are using with a **prefix**:

Written	Prefix means	Value
42	no prefix — plain decimal	forty-two
<code>0b101010</code>	<code>0b</code> — read the digits as binary	forty-two
<code>0x2A</code>	<code>0x</code> — read the digits as hexadecimal	forty-two

All three are the same value to the compiler. You pick whichever makes the thing you are doing readable: decimal for counting, binary for seeing bits, hex for writing bytes compactly.

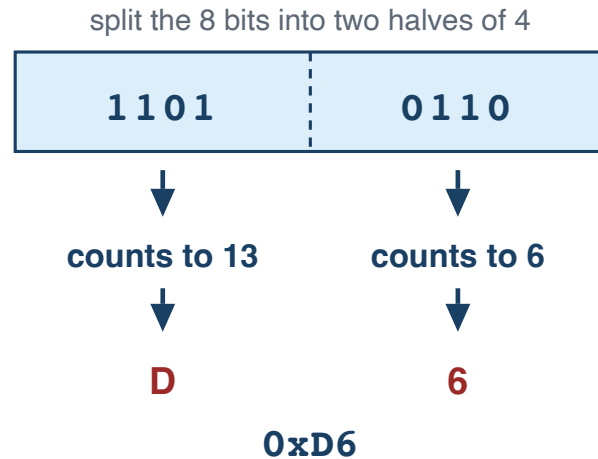
Note

`0b` is not standard until C23. GCC and Clang accept it, and these slides use it to show bit patterns, but in ISO C17 write `0x2A` or `42`.

Why Hex Fits Bytes So Well

Eight ones and zeros is easy to miscount. Four bits count 0–15 — exactly one hex digit — so one byte is always two digits, and you can still read the bits straight off the page.

One byte = two hex digits



**4 bits count 0 to 15,
so we need 6 extra digits:**

10 = A
11 = B
12 = C
13 = D
14 = E
15 = F

0 through 9 stay as they are.

Characters Are Just Numbers

The rule that turns 65 into **A** is **ASCII** — the American Standard Code for Information Interchange — a table agreed in the 1960s that gives every character a number from 0 to 127.

Characters	Codes	Worth knowing
'0'-'9'	48–57 (0x30–0x39)	consecutive, so <code>c - '0'</code> gives the digit
'A'-'Z'	65–90 (0x41–0x5A)	consecutive
'a'-'z'	97–122 (0x61–0x7A)	exactly 32 more than the capitals

```
char c = '7';  
int d = c - '0';           // 7  
int is_digit = (c >= '0' && c <= '9'); // 1
```

In C the literal **'A'** is the number 65. Nothing clever is happening.

The Whole Table

ASCII in full — every code from 0 to 127

Read down a column. Grey are control codes with no glyph; the tinted blocks are the digits, the capitals and the lowercase letters.

0	00	NUL	16	10	DLE	32	20	SP	48	30	0	64	40	@	80	50	P	96	60	`	112	70	p
1	01	SOH	17	11	DC1	33	21	!	49	31	1	65	41	A	81	51	Q	97	61	a	113	71	q
2	02	STX	18	12	DC2	34	22	"	50	32	2	66	42	B	82	52	R	98	62	b	114	72	r
3	03	ETX	19	13	DC3	35	23	#	51	33	3	67	43	C	83	53	S	99	63	c	115	73	s
4	04	EOT	20	14	DC4	36	24	\$	52	34	4	68	44	D	84	54	T	100	64	d	116	74	t
5	05	ENQ	21	15	NAK	37	25	%	53	35	5	69	45	E	85	55	U	101	65	e	117	75	u
6	06	ACK	22	16	SYN	38	26	&	54	36	6	70	46	F	86	56	V	102	66	f	118	76	v
7	07	BEL	23	17	ETB	39	27	'	55	37	7	71	47	G	87	57	W	103	67	g	119	77	w
8	08	BS	24	18	CAN	40	28	(	56	38	8	72	48	H	88	58	X	104	68	h	120	78	x
9	09	HT	25	19	EM	41	29	)	57	39	9	73	49	I	89	59	Y	105	69	i	121	79	y
10	0A	LF	26	1A	SUB	42	2A	*	58	3A	:	74	4A	J	90	5A	Z	106	6A	j	122	7A	z
11	0B	VT	27	1B	ESC	43	2B	+	59	3B	;	75	4B	K	91	5B	[	107	6B	k	123	7B	{
12	0C	FF	28	1C	FS	44	2C	,	60	3C	<	76	4C	L	92	5C	\	108	6C	l	124	7C	
13	0D	CR	29	1D	GS	45	2D	-	61	3D	=	77	4D	M	93	5D	]	109	6D	m	125	7D	}
14	0E	SO	30	1E	RS	46	2E	.	62	3E	>	78	4E	N	94	5E	^	110	6E	n	126	7E	~
15	0F	SI	31	1F	US	47	2F	/	63	3F	?	79	4F	O	95	5F	_	111	6F	o	127	7F	DEL

Each cell is decimal, hex, then the character. 'a' - 'A' = 32 — one bit apart.

And Above 127?

128 to 255: not ASCII, and not one answer

These are the ISO 8859-1 (Latin-1) meanings. Windows-1252 disagrees about 128–159.

In UTF-8 — what your files almost certainly are — none of these bytes is a character on its own.

128 80 ctl	144 90 ctl	160 A0	176 B0 °	192 C0 À	208 D0 Ð	224 E0 à	240 F0 ð
129 81 ctl	145 91 ctl	161 A1 í	177 B1 ±	193 C1 Á	209 D1 Ñ	225 E1 á	241 F1 ñ
130 82 ctl	146 92 ctl	162 A2 ¢	178 B2 ²	194 C2 Â	210 D2 Ò	226 E2 â	242 F2 ò
131 83 ctl	147 93 ctl	163 A3 £	179 B3 ³	195 C3 Ã	211 D3 Ó	227 E3 ã	243 F3 ó
132 84 ctl	148 94 ctl	164 A4 ¤	180 B4 ´	196 C4 Ä	212 D4 Ô	228 E4 ä	244 F4 ô
133 85 ctl	149 95 ctl	165 A5 ¥	181 B5 µ	197 C5 Å	213 D5 Õ	229 E5 å	245 F5 õ
134 86 ctl	150 96 ctl	166 A6 ¦	182 B6 ¶	198 C6 Æ	214 D6 Ö	230 E6 æ	246 F6 ö
135 87 ctl	151 97 ctl	167 A7 §	183 B7 ·	199 C7 Ç	215 D7 ×	231 E7 ç	247 F7 ÷
136 88 ctl	152 98 ctl	168 A8 ¨	184 B8 ¸	200 C8 È	216 D8 Ø	232 E8 è	248 F8 ø
137 89 ctl	153 99 ctl	169 A9 ©	185 B9 ¹	201 C9 É	217 D9 Ù	233 E9 é	249 F9 ù
138 8A ctl	154 9A ctl	170 AA ª	186 BA º	202 CA Ê	218 DA Ú	234 EA ê	250 FA ú
139 8B ctl	155 9B ctl	171 AB «	187 BB »	203 CB Ë	219 DB Û	235 EB ë	251 FB û
140 8C ctl	156 9C ctl	172 AC ¬	188 BC ¼	204 CC Ì	220 DC Ü	236 EC ì	252 FC ü
141 8D ctl	157 9D ctl	173 AD	189 BD ½	205 CD Í	221 DD Ý	237 ED í	253 FD ý
142 8E ctl	158 9E ctl	174 AE ®	190 BE ¾	206 CE Î	222 DE Þ	238 EE î	254 FE þ
143 8F ctl	159 9F ctl	175 AF ¯	191 BF º	207 CF Ï	223 DF ß	239 EF ï	255 FF ÿ

A byte above 127 means nothing until you know the encoding it was written in.

Ñ is 209 here and ñ is 241 — but in UTF-8 each one takes **two** bytes, so "Peña" is 5 bytes, not 4.

ASCII stops at 127 — it only ever needed 7 bits. Everything above that is a *different standard's* idea of what the byte means.

So – Can We Type Spanish?

Put the two tables together and you have 256 slots. Is that enough for every character Spanish actually needs?

Every character Spanish needs has a slot

Blue came from the ASCII table. Orange came from the half above 127.

¿	C	u	á	n	t	o	s	_	a	ñ	o	s	?
191	67	117	225	110	116	111	115	32	97	241	111	115	63

14 characters, 14 bytes — one byte each, and nothing is missing.

ñ Ñ á é í ó ú Á É Í Ó Ú ü Ü ¿ ¡ — all sixteen live in the upper half.

! Important

Yes — but only because someone chose Latin-1. That same byte 241 is *ń* in Latin-2, *Я* in KOI8-R and *ρ* in Greek, and nothing in the file says which. Spanish and Polish cannot share one 8-bit document at all. That is what UTF-8 fixed, at the price that *ñ* takes two bytes — so `strlen("Peña")` is 5.

How Does C Know What to Print?

The bits never say what they are. `printf` prints whatever the **format code** asks for — so the same `x` comes out as a number or as a character:

```
int x = 65;
printf("%d\n", x);      // 65
printf("%c\n", x);      // A

unsigned a = 12, b = 10;
printf("%u\n", a & b);  // 8
printf("%u\n", a << 2); // 48
```

Nothing about `x` changed between those first two lines. The only thing that changed is what you told `printf` to do with it.

Telling printf What You Handed It

`printf` cannot see the type of what you pass — the format string is how it finds out. Get that wrong and the output is garbage, or worse:

Code	Prints
<code>%d</code>	a signed <code>int</code>
<code>%u</code>	an <code>unsigned</code>
<code>%c</code>	one character
<code>%s</code>	a string (<code>char *</code>)
<code>%zu</code>	a <code>size_t</code> , as from <code>sizeof</code>

Code	Prints
<code>%x %X</code>	hex, lower or upper case
<code>%f</code>	a <code>double</code>
<code>%p</code>	a pointer — cast it to <code>void *</code>
<code>%%</code>	a literal <code>%</code> sign
<code>%02X</code>	hex, padded to 2 with zeros

Warning

Build with `-Wall` and the compiler catches most mismatches — `%s` with an `int`, `%d` with a `double`. It stays quiet about `%d` with an `unsigned`, though, because they are the same width. That one is still wrong, and it is the one you will write.

Bit-Level Operations

For these bit patterns, C provides six bitwise operators:

Op	Name	Example
<code>&</code>	AND	<code>0b1100 & 0b1010 = 0b1000</code>
<code>\ </code>	OR	<code>0b1100 \ 0b1010 = 0b1110</code>
<code>^</code>	XOR(Exclusive OR)	<code>0b1100 ^ 0b1010 = 0b0110</code>
<code>~</code>	NOT	<code>~0b1100 = 0b1111...11110011</code> (every bit flips, including the leading zeros)
<code><<</code>	Left shift	<code>0b0011 << 2 = 0b1100</code>
<code>>></code>	Right shift	<code>0b1100 >> 2 = 0b0011</code>

Warning

Right shift of signed integers is **implementation-defined** in C. GCC uses **arithmetic** right shift (preserves sign bit). Don't rely on this unless you're sure of your compiler.

Bitwise Operations, One Column at a Time

The same two inputs, three separate rules

Every output bit is decided on its own, from just the two bits directly above it.

INPUTS

a	1	1	0	0	= 1100
b	1	0	1	0	= 1010

a & b	1	0	0	0	AND = 1000
-------	---	---	---	---	------------

1 only where both are 1

a b	1	1	1	0	OR = 1110
-------	---	---	---	---	-----------

1 where either is 1

a ^ b	0	1	1	0	XOR = 0110
-------	---	---	---	---	------------

1 where they differ

De Morgan's Laws

Two rules let you rewrite any mix of $\&$, $|$ and $\sim$ — and they are how Lab 1 asks you to build $\wedge$ out of nothing but $\sim$ and $\&$.

Flip the whole thing, and AND becomes OR

$$\sim(x \ \& \ y) == \sim x \ | \ \sim y$$

$$\sim(x \ | \ y) == \sim x \ \& \ \sim y$$

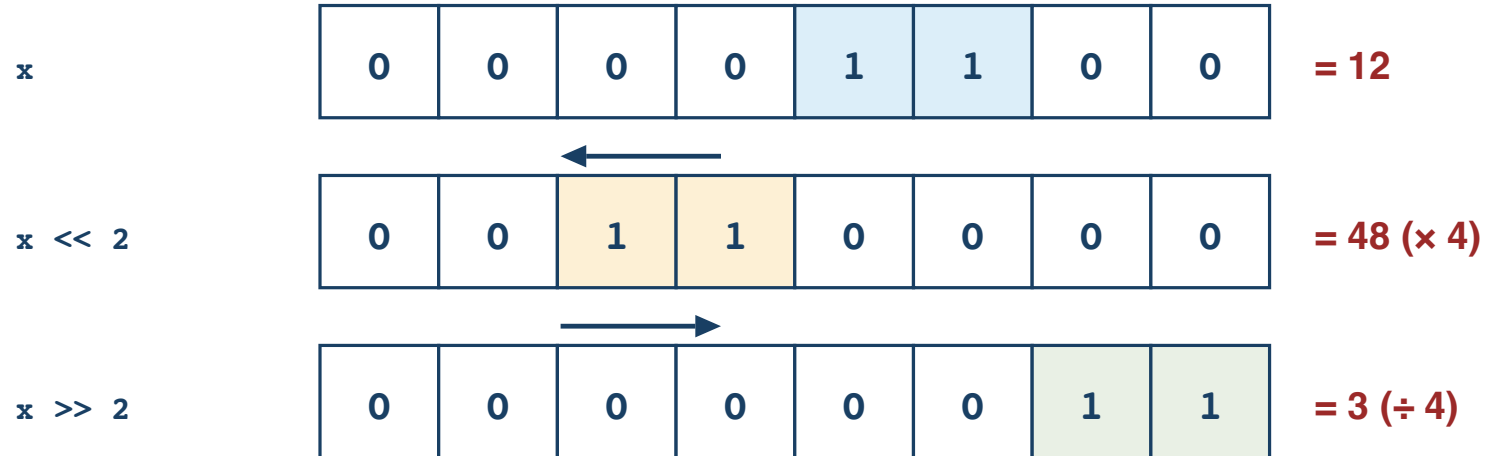
x	y	x & y	$\sim(x \ \& \ y)$	$\sim x \ \ \sim y$
0	0	0	1	1
0	1	0	1	1
1	0	0	1	1
1	1	1	0	0

same in all four rows

Two inputs give four possible rows, so checking all four is a complete proof.

Shifting Bits

Shifting slides every bit sideways

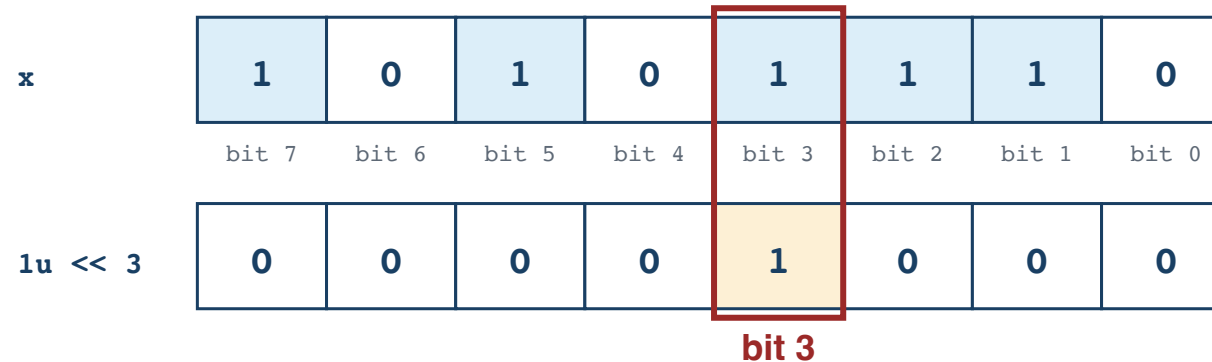


Bits pushed off the end are gone for good; zeros come in behind them.

Shifting left by n multiplies by 2^n . Shifting an unsigned value right by n divides by 2^n .

Test, Set, Clear, Toggle

`1u << 3` builds a mask with exactly one bit set



Bit 0 is the rightmost — the ones place. The mask is 0 everywhere except the bit you want.

```
unsigned x = 0xAE;           // 10101110
x |= (1u << 3);              // SET    – bit 3 becomes 1
x &= ~(1u << 3);             // CLEAR  – bit 3 becomes 0
x ^= (1u << 3);              // TOGGLE  – bit 3 flips
if (x & (1u << 3)) { }      // TEST   – nonzero when bit 3 is 1
```

Those four are most of Lab 1. Keep the value **unsigned** — shifting into a sign bit is undefined.

What Two's Complement Means

Complement means the part that completes a whole. Flip every bit of x to get its *one's complement*; add 1 to that and you have its **two's complement**, which is how C writes $-x$:

To write -5 in 8 bits

1. Start with +5

00000101

↓ flip every 0 ↔ 1

2. One's complement

11111010

+ 1

↓

3. Two's complement

11111011

= -5

More 8-bit examples

+1: 00000001

-1: 11111111

0: 00000000

two's: 00000000

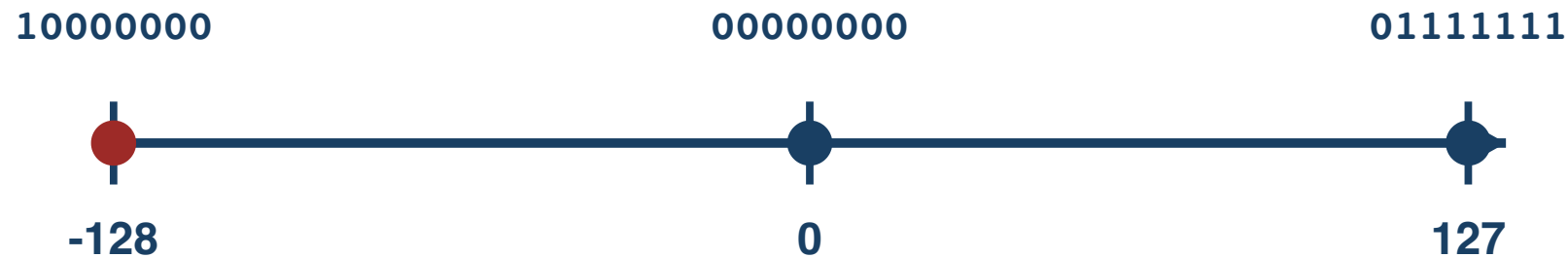
Zero stays zero.

A ninth bit is simply discarded — 8 bits keep only their rightmost 8.

The Range It Buys You

An N-bit two's complement covers $-2^{(N-1)}$ to $2^{(N-1)} - 1$:

8-bit two's-complement values



There is one more negative value than positive value.

$TMin = -TMax - 1$ — there is no positive 128 in 8 bits, so negating $TMin$ overflows and is undefined.

Sign Extension

When converting a smaller signed type to a larger one, copy the sign bit:

```
int8_t  x = -1;    // 0b11111111  
int16_t y = x;     // 0b1111111111111111 = -1 ✓
```

Copy the sign bit into the new high bits

int8_t -1

11111111

extend



int16_t -1

1111111111111111

Truncation

When converting larger to smaller, drop the high bits:

```
int16_t x = 257;  
uint8_t y = x;    // 1: conversion to unsigned is modulo 256
```

Keep the low bits; discard the high bits

int16_t 257

000000001

000000001



keep these low 8 bits

int8_t result = 00000001 = 1

Size of Integer Types

```
printf("char:      %zu\n", sizeof(char));      // 1 C byte (not always 8 bits)
printf("short:     %zu\n", sizeof(short));     // 2
printf("int:       %zu\n", sizeof(int));       // 4
printf("long:      %zu\n", sizeof(long));      // 8 (LP64)
printf("long long: %zu\n", sizeof(long long)); // 8
printf("pointer:   %zu\n", sizeof(void*));     // 8
```



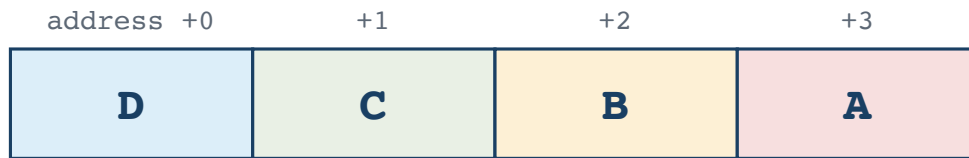
Tip

Use `sizeof` and fixed-width types (`int32_t`, `uint64_t`) when exact sizes matter. Never assume `int` is 4 bytes; use `CHAR_BIT` when bit width matters. Plain `char` is its own trap: whether it is signed is implementation-defined, so say `signed char` or `unsigned char` when it matters.

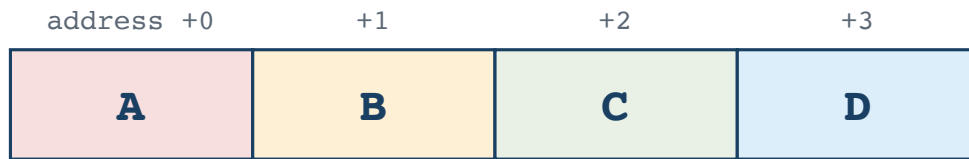
Byte Ordering (Endianness)

An `int` occupies 4 consecutive bytes, so those 4 bytes have to be laid out in some order. **Endianness** is that order. One question matters: **which byte goes at the lowest address?**

Little-endian: D is first in memory



Big-endian: A is first in memory



```
int x = 1;    // ask the machine: which byte of x sits at the lowest address?
if (*(unsigned char *)&x == 1) printf("Little-endian\n");
else                               printf("Big-endian\n");
```


AI Systems Connection

Quantization is one important reason large models can fit on more hardware. A common symmetric int8 scheme represents a block of weights with 8-bit signed values and a scale factor:

$$\text{real_value} \approx \text{scale} \times \text{int8_value} \qquad \text{scale} = \text{max_abs} / 127$$

Everything today shows up here. Two int8 values need 16 bits for their product, so a long dot product accumulates into something wider — often `int32_t`. And if a signed weight is loaded as *unsigned* into a wider register, sign extension does not happen, negatives become huge positives, and the answer is quietly wrong.

Try it: quantize a float array with `scale = max_abs / 127`, dequantize, and measure the error.

Practice

1. Write `0xD6` in binary, and `10110010` in hex, without using a computer.
2. What does `x & 1` tell you about `x`? And what does `x & (x - 1)` do? Try both on `0b0110` by hand.
3. Write one expression that clears bit 5 of `unsigned x`, and one that tests whether bit 0 and bit 7 are both set.
4. What is `TMax` for a 32-bit `int`? What is `TMin`?
5. Write a C expression that checks if your machine is little-endian.
6. Is left-shifting a 32-bit `int` by 32 positions defined in C? Why not?
7. Why is `-TMin` not representable in the same signed type?

Reading

- CS:APP §2.1 (Information Storage)
- CS:APP §2.2 (Integer Representations)